

Método BPO – Site + Painel de Controle com Firebase

Você recebeu 3 arquivos:

Arquivo	Vai para onde	O que é
<code>dados.js</code>	GitHub (junto do site)	Configuração do Firebase + o "banco de dados" padrão do site.
<code>index.html</code>	GitHub (junto do dados.js)	O seu site, agora buscando os textos do Firebase em vez de ter tudo fixo no código.
<code>painel.html</code>	Fica só com o cliente (não precisa subir pro GitHub, mas pode)	O painel de controle remoto para editar o site.

Os três arquivos **precisam ficar na mesma pasta** (mesmo repositório), porque `index.html` e `painel.html` importam o `dados.js` com um caminho relativo:

```
import { ... } from './dados.js';
```

Se algum dia hospedar o painel em outro lugar separado do site, é só ajustar esse caminho para a URL completa do `dados.js` publicado.

Como funciona

1. Todo o conteúdo editável do site (títulos, textos, cards, preço, depoimentos, FAQ) fica guardado em **um único documento** no Firestore: coleção `site`, documento `conteudo`.
2. `index.html` **escuta esse documento em tempo real**. Assim que algo muda no Firestore, o site atualiza sozinho — sem precisar dar F5 nem fazer novo deploy.
3. `painel.html` é o "controle remoto": ele mostra formulários para editar cada parte do site e, ao clicar em **Salvar**

alterações, grava tudo de volta no mesmo documento do Firestore.

4. Se o Firestore ainda estiver vazio (primeiro uso) ou ficar indisponível, o site cai automaticamente para o conteúdo padrão (que é uma cópia exata do texto que já existia), então ele nunca fica "quebrado" ou em branco.

Passo 1 — Configurar o Firestore

1. No [Console do Firebase](#), abra o projeto `marcusviniciuscliente`.
2. Vá em **Firestore Database** → se ainda não existir, clique em **Criar banco de dados** (modo produção).
3. Nas **Regras** (aba "Regras"), use algo assim para começar:

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /site/conteudo {
      allow read: if true;           //
    }
    // qualquer visitante pode LER (o site precisa disso)
    allow write: if true;           // ⚠️
  }
}
```

ver observação abaixo sobre segurança

```
}  
}  
}
```

⚠ **Sobre segurança:** a regra `allow write: if true` permite que **qualquer pessoa que descubra a configuração do Firebase** (que fica visível no código do site, isso é normal) consiga alterar o conteúdo. A senha do `painel.html` é só uma proteção visual simples — ela não impede alguém de escrever diretamente na API do Firestore. Para produção, recomendo ativar o **Firebase Authentication** (por exemplo, login por e-mail/senha só para o cliente) e trocar a regra de escrita para algo como:
`allow write: if request.auth != null; .`
Posso te ajudar a configurar isso se quiser — é um passo a mais, mas deixa o painel realmente protegido.

4. Não precisa criar o documento manualmente: na primeira vez que o cliente clicar em **Salvar alterações** no painel, o documento `site/conteudo` é criado automaticamente com todo o conteúdo.
-

Passo 2 — Subir para o GitHub

1. Crie (ou use) um repositório com `index.html` e `dados.js` na raiz (ou na mesma pasta).
2. Se for usar **GitHub Pages**, ative em Settings → Pages, apontando para a branch/pasta onde estão os arquivos.
3. `painel.html` **não precisa** estar público — mas se quiser subir junto (por exemplo numa pasta `/admin`), funciona normalmente, só que qualquer pessoa com o link e a senha vai conseguir acessá-lo. Se preferir mais discrição, hospede `painel.html` separadamente (ou dê um nome de arquivo difícil de adivinhar) e envie o link só para o cliente.

Passo 3 — Entregar o painel para o cliente

- Abra `painel.html` (localmente ou publicado) e defina a senha de acesso editando esta linha dentro do próprio arquivo:

```
const SENHA_PAINEL = "bpo2026";
```

Troque "bpo2026" pela senha que quiser antes de entregar.

- No painel, o cliente consegue:
 - Editar o título, subtítulo, selos e botão do topo (Hero).
 - Editar os 4 números de prova social (estatísticas).
 - Editar título/descrição e **adicionar, editar ou remover** cards de:
"O que você vai aprender", "Perfil do aluno ideal", "O que está incluso",
"Nossos diferenciais".
 - Editar o texto da garantia.
 - Editar preço, desconto, parcelamento e o link de pagamento (Hotmart).
 - **Adicionar, editar ou remover** depoimentos (incluindo as mensagens de cada conversa simulada de WhatsApp).
 - **Adicionar, editar ou remover** perguntas do FAQ.
 - Clicar em **Salvar alterações** para publicar tudo em tempo real.
-

Limitações conhecidas (para não ter surpresa)

- **Ícones dos cards são automáticos.** O painel não deixa escolher o ícone de cada card — eles giram automaticamente entre um conjunto fixo, na ordem em que os cards aparecem. Dá pra evoluir isso depois se quiser escolher ícone por ícone.
- **Layout e cores continuam fixos no HTML.** O painel edita **conteúdo** (texto, preço, cards, depoimentos), não o design/CSS da página.
- **A senha do painel é local ao navegador**, não é uma autenticação real — veja a observação de segurança no Passo 1 se quiser reforçar isso.
- Campos de "quantidade fixa" (os 4 números de estatística e os 4 selos do topo) não têm botão de adicionar/remover — são sempre 4, só o texto muda.